Rockchip IDC Developer Tutorial

ID: RK-KF-SF-049

Release Version: V1.1.1

Release Date: 2026-04-21

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY.

THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Product Version

Product Name	Product Version
RK3588/RK3576/RK3572/RK3562	ALL

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	Jieke Chen	2025-04-01	Initial version
V1.0.1	Jieke Chen	2025-10-30	Initialization parameters support specifying the OpenCL/Mali library path.
V1.1.0	Jieke Chen	2026-02-26	Update description for Mesh.
V1.1.1	Jieke Chen	2026-04-21	Support RK3572

Contents

Rockchip IDC Developer Tutorial

- 1. Overview
  - 1.1 Version Number Specification
  - 1.2 Main Function List
- 2. Application Interface Specifications
  - 2.1 LUT
    - 2.1.1 RKALG_IDC_LUT_GetVersion
    - 2.1.2 RKALG_IDC_LUT_Init
    - 2.1.3 RKALG_IDC_LUT_DoLut
    - 2.1.4 RKALG_IDC_LUT_Deinit
- 3. Parameter Definitions
  - 3.1 RKALG_IDC_IMG_FMT_E
  - 3.2 RKALG_IDC_MESH_TYPE_E
  - 3.3 RKALG_IDC_IMAGE_S
  - 3.4 RKALG_IDC_MESH_S
  - 3.5 RKALG_LUT_CTX_S
  - 3.6 RKALG_LUT_MODE_E
  - 3.7 RKALG_LUT_OP_E
  - 3.8 RKALG_LUT_INIT_PARAMS_S
  - 3.9 RKALG_LUT_OP_ATTR_S
  - 3.10 RKALG_LUT_TASK_S
- 4. API Calling Example

1. Overview

This document provides a tutorial on the use of the IDC Algorithm, including a description of the API, parameter definitions, and calling example. For an introduction to the basic principles and functions of IDC, see “Rockchip_IDC_Developer_Guide_EN.md”.

1.1 Version Number Specification

Version format

```
IDC app Version: major.minor.revision.subversion date:YYYY-MM-DD
```

Where `[app]` represents the module name (e.g., LUT).

Example

```
IDC LUT Version: 1.0.1.66 date: 2025-03-04
```

1.2 Main Function List

Key API interfaces of LUT App:

Function Name	Description
RKALG_IDC_LUT_GetVersion	Gets LUT application version info
RKALG_IDC_LUT_Init	Initializes LUT application resources
RKALG_IDC_LUT_DoLut	Executes image remapping tasks
RKALG_IDC_LUT_Deinit	Releases LUT application resources

2. Application Interface Specifications

2.1 LUT

2.1.1 RKALG_IDC_LUT_GetVersion

Function: Retrieves version information of the LUT application interface.

Definition:

```
void RKALG_IDC_LUT_GetVersion(char idcLutVersion[128]);
```

Parameters:

Parameter	Type	Description
idcLutVersion	output	Buffer for version string

Notes: None **Return:** None

2.1.2 RKALG_IDC_LUT_Init

Function: Initializes LUT application context for image remapping.

Definition:

```
int RKALG_IDC_LUT_Init(RKALG_LUT_CTX_S *pLutCtx, RKALG_LUT_INIT_PARAMS_S *pInitParams);
```

Parameters:

Parameter	Type	Description
pLutCtx	output	LUT context structure pointer
pInitParams	input	Initialization parameters

Notes:

- First initialization takes longer if no cached OpenCL program exists
- Default cache path: `/data/` (configurable path coming soon)

Return: 0=success, non-zero=failure

2.1.3 RKALG_IDC_LUT_DoLut

Function: Executes image remapping tasks (input image + output image + mesh table + operator attributes).

Definition:

```
int RKALG_IDC_LUT_DoLut(RKALG_LUT_CTX_S *pLutCtx, RKALG_LUT_TASK_S *pTask);
```

Parameters:

Parameter	Type	Description
pLutCtx	input	LUT context structure pointer
pTask	input	Task parameters

Notes:

- Re-initialization required if resolution changes.
- Auto-mode performance stats saved as `.csv` in `/data/` (configurable path coming soon)

Return: 0=success, non-zero=failure

2.1.4 RKALG_IDC_LUT_Deinit

Function: Destroys LUT context and releases resources.

Definition:

```
int RKALG_IDC_LUT_Deinit(RKALG_LUT_CTX_S *pLutCtx);
```

Parameters:

Parameter	Type	Description
pLutCtx	input	LUT context structure pointer

Return: 0=success, non-zero=failure

3. Parameter Definitions

3.1 RKALG_IDC_IMG_FMT_E

Description: Supported image formats (currently NV12/YUV422SP only)

Definition:

```
typedef enum _rkAlgIDC_IMG_FMT_E {
    RKALG_IDC_IMG_FMT_NV12 = 0,
    RKALG_IDC_IMG_FMT_GRAY = 1
} RKALG_IDC_IMG_FMT_E;
```

3.2 RKALG_IDC_MESH_TYPE_E

Description: Mesh table types.

Definition:

```
typedef enum _rkAlgIDC_MESH_TYPE_E {
    RKALG_IDC_MESH_TYPE_COMMON = 0, /* meshX and meshY, for GPU */
    RKALG_IDC_MESH_TYPE_MERGED = 1, /* meshXy, for GPU */
    RKALG_IDC_MESH_TYPE_FEC    = 2  /* for FEC */
} RKALG_IDC_MESH_TYPE_E;
```

Note: Currently only `RKALG_IDC_MESH_TYPE_MERGED` is supported.

3.3 RKALG_IDC_IMAGE_S

Description: Single frame image information structure.

Definition:

```
typedef struct _rkAlgIDC_IMAGE_S {
    uint64_t id;
    RKALG_IDC_IMG_FMT_E eImgFmt;
    uint32_t u32Width;
    uint32_t u32Height;
    uint32_t u32Stride[3];      // (Y/U/V channels)
    uint32_t u32HgtStride[3];
    int fd[3];
    void *virAddr[3];
    void *phyAddr[3]; // reserved
} RKALG_IDC_IMAGE_S;
```

Note: Physical address fields are reserved for future use.

3.4 RKALG_IDC_MESH_S

Description: Mesh table information structure.

Definition:

```
typedef struct {
    uint64_t id;                // Identifier
    uint32_t u32StepX;          // X sampling step (pixels)
    uint32_t u32StepY;          // Y sampling step (pixels)
    uint32_t u32Width;          // X sample points
    uint32_t u32Height;         // Y sample points
    uint32_t u32Stride;         // Width stride
    uint32_t u32HgtStride;      // Height stride
    RKALG_IDC_MESH_TYPE_E eMeshType;
    int fd[4];                  // DRM file descriptors
    void *virAddr[4];           // Virtual addresses
    void *phyAddr[4];           // Physical addresses (reserved)
} RKALG_IDC_MESH_S;
```

Note:

1. Physical address fields are reserved for future use.
2. Sampling step is usually a power of 2 and the minimum value should be 4.
3. meshWidth needs to be at least ((dst image width-1)/stepX +2), meshHeight needs to be at least ((dst image height-1)/stepY + 2).
4. Refer to the sample code below for usage.

3.5 RKALG_LUT_CTX_S

Description: LUT application context pointer structure.

Definition:

```
typedef struct _rkAlgLutCtx_S {
    void *p;
} RKALG_LUT_CTX_S;
```

3.6 RKALG_LUT_MODE_E

Description: LUT application modes.

Definition:

```
typedef enum _rkAlgLutMode_E {
    RKALG_IDC_LUT_DEFAULT_MODE = 0, /* Use default Kernel and NDRange */
    RKALG_IDC_LUT_MANUAL_MODE  = 1, /* Use user defined Kernel and NDRange */
    RKALG_IDC_LUT_AUTO_MODE    = 2  /* Auto Detect best Kernel and NDRange. This takes some
extra time at startup*/
} RKALG_LUT_MODE_E;
```

3.7 RKALG_LUT_OP_E

Description: Remapping operators.

Definition:

```
typedef enum _rkAlgLutOp_E {
    RKALG_IDC_LUT_OP_0 = 0, /* default op*/
    RKALG_IDC_LUT_OP_1 = 1,
    RKALG_IDC_LUT_OP_2 = 2,
    RKALG_IDC_LUT_OP_3 = 3
} RKALG_LUT_OP_E;
```

3.8 RKALG_LUT_INIT_PARAMS_S

Description: LUT initialization parameters.

Definition:

```
typedef struct _rkAlgLutInitParams_S {
    uint32_t u32SrcWidth;     /* 2 pixel aligned */
    uint32_t u32SrcHeight;
    uint32_t u32SrcStride;    /* 64 pixel aligned */
    uint32_t u32SrcHgtStride; /* 2 pixel aligned */
    uint32_t u32DstWidth;
    uint32_t u32DstHeight;    /* 4 pixel aligned */
    uint32_t u32DstStride;    /* 16 pixel aligned */
    uint32_t u32DstHgtStride; /* 8 pixel aligned */
    uint32_t u32Affinity;
    char     cOpenclLibraryPath[256]; /* (optional) OpenCL/mali Library Path. If missing, the
default system library path will be used. */
    RKALG_LUT_MODE_E eMode;
} RKALG_LUT_INIT_PARAMS_S;
```

Note:

1. Alignment Requirements: srcStride: 64 pixel aligned, srcHeightStride: 2 pixel aligned. dstStride: 16 pixel aligned, dstHeightStride: 8 pixel aligned.
2. If set to manual mode, operator attributes need to be configured when calling the DoLut API.

3.9 RKALG_LUT_OP_ATTR_S

Description: Remapping operator attributes.

Definition:

```
typedef struct {
    RKALG_LUT_OP_E eOp;        // Operator type (manual mode only)
    uint32_t u32LocalSize[2];  // Thread dimensions (manual mode only)
} RKALG_LUT_OP_ATTR_S;
```

3.10 RKALG_LUT_TASK_S

Description: Remapping task definition (input + output + LUT + optional attributes).

Definition:

```
typedef struct _rkAlgLutTask_S {
    RKALG_IDC_IMAGE_S   *pSrcImage; /* src attr and data */
    RKALG_IDC_IMAGE_S   *pDstImage; /* dst attr and data */
    RKALG_IDC_MESH_S    *pMesh;     /* mesh attr and data */
    RKALG_LUT_OP_ATTR_S *pOpAttr;   /* op attr, only work in manual mode.*/
} RKALG_LUT_TASK_S;
```

Note:

1. Mesh: Currently only merged floating-point type supported (X/Y coordinates interleaved).
2. Images: Only YUV420SP format with specific alignment supported.

   For src image, stride should be 64-pixel aligned, hgtStride should be 2-pixel aligned. For dst image, stride should be 16-pixel aligned, hgtStride should be 8-pixel aligned. For src and dst both, only first channel of following params need to be configured: stride, hgtStride, virtual address, fd.

4. API Calling Example

LUT sample code, implementing image remapping via mesh.

Key Notes:

1. Interleaved(merged) floating-point mesh required.
2. Supports dynamic LUT updates without reinitialization if resolution unchanged.
3. The log level can be adjusted via the `RKALG_IDC_LOG_LEVEL` environment variable, with a valid range of [0, 7] (default is 4). Higher values enable progressively more verbose logging output. For example:

   ```
   export RKALG_IDC_LOG_LEVEL=5
   ```

Example Code:

```
void test()
{
    int ret = 0;
    printf("test begin.\n");

    char version[128];
    RKALG_IDC_LUT_GetVersion(version);
    printf("%s\n", version);

    /* Step 1-1. Config initial params */
```

```
    RKALG_LUT_INIT_PARAMS_S stInitParams;
    stInitParams.u32SrcWidth  = 3840;
    stInitParams.u32SrcHeight = 2160;
    stInitParams.u32SrcStride = alignUp(stInitParams.u32SrcWidth, 64);
    stInitParams.u32SrcHgtStride = alignUp(stInitParams.u32SrcHeight, 2);
    stInitParams.u32DstWidth  = 3840;
    stInitParams.u32DstHeight = 2160;
    stInitParams.u32DstStride = alignUp(stInitParams.u32DstWidth, 16);
    stInitParams.u32DstHgtStride = alignUp(stInitParams.u32DstHeight, 8);
    stInitParams.eMode        = RKALG_IDC_LUT_DEFAULT_MODE;
    sprintf(stInitParams.cOpenclLibraryPath, "/usr/lib/libmali.so");

    char dst_img_path[256];
    char src_img_path[256];
    sprintf(src_img_path, TMP_DATA_PATH"input_%dx%d_nv12.yuv", stInitParams.u32SrcWidth,
stInitParams.u32SrcHeight);
    sprintf(dst_img_path, TMP_DATA_PATH"output_%dx%d_nv12.yuv", stInitParams.u32DstWidth,
stInitParams.u32DstHeight);
    char meshxy_path[] = TMP_DATA_PATH"/rkalg_idc_gpu_meshxy00.bin";

    /* Step 1-2. Create LUT context */
    RKALG_LUT_CTX_S stLutCtx;
    RKALG_IDC_LUT_Init(&stLutCtx, &stInitParams);

    /* Step 2-1. malloc DMA Buffer for srcImg and read data */
    RKALG_IDC_IMAGE_S srcImage;
    memset(&srcImage, 0, sizeof(RKALG_IDC_IMAGE_S));
    srcImage.eImgFmt         = RKALG_IDC_IMG_FMT_NV12; // only NV12 supported now
    srcImage.u32Width        = stInitParams.u32SrcWidth;
    srcImage.u32Height       = stInitParams.u32SrcHeight;
    srcImage.u32Stride[0]    = stInitParams.u32SrcStride;
    srcImage.u32HgtStride[0] = stInitParams.u32SrcHgtStride;

    DmaBufferInfo dmaBufSrc;
    memset(&dmaBufSrc, 0, sizeof(DmaBufferInfo));
    int bpp_nv12 = 12 * sizeof(unsigned char);
    alloc_drm_buf(&dmaBufSrc, srcImage.u32Stride[0], srcImage.u32HgtStride[0], bpp_nv12);
    srcImage.fd[0]      = dmaBufSrc.fd;
    srcImage.virAddr[0] = dmaBufSrc.ptr;

    size_t srcImgSize = srcImage.u32Width * srcImage.u32Height * 3 / 2 * sizeof(unsigned
char);
    ret |= readBinaryFile(src_img_path, srcImage.virAddr[0], srcImgSize);
    if (ret != 0) {
        printf("read src image failed.\n");
        return;
    }

    /* Step 2-2. malloc DMA Buffer for dstImg and config */
    RKALG_IDC_IMAGE_S dstImage;
    memset(&dstImage, 0, sizeof(RKALG_IDC_IMAGE_S));
    dstImage.eImgFmt         = RKALG_IDC_IMG_FMT_NV12; // Only support NV12 now
    dstImage.u32Width        = stInitParams.u32DstWidth;
```

```
    dstImage.u32Height       = stInitParams.u32DstHeight;
    dstImage.u32Stride[0]    = stInitParams.u32DstStride;
    dstImage.u32HgtStride[0] = stInitParams.u32DstHgtStride;

    DmaBufferInfo dmaBufDst;
    memset(&dmaBufDst, 0, sizeof(DmaBufferInfo));
    alloc_drm_buf(&dmaBufDst, dstImage.u32Stride[0], dstImage.u32HgtStride[0], bpp_nv12);
    dstImage.fd[0]      = dmaBufDst.fd;
    dstImage.virAddr[0] = dmaBufDst.ptr;

    /* Step 2-3. malloc DMA Buffer for Xy mesh and read data */
    RKALG_IDC_MESH_S mesh;
    memset(&mesh, 0, sizeof(RKALG_IDC_MESH_S));
    mesh.u32StepX     = 32;
    mesh.u32StepY     = 16;
    mesh.u32Width     = (dstImage.u32Stride[0] - 1) / mesh.u32StepX + 2;
    mesh.u32Height    = (dstImage.u32Height - 1) / mesh.u32StepY + 2;
    mesh.u32Stride    = mesh.u32Width;
    mesh.u32HgtStride = mesh.u32Height;
    mesh.eMeshType    = RKALG_IDC_MESH_TYPE_MERGED;

    DmaBufferInfo dmaBufMesh;
    memset(&dmaBufMesh, 0, sizeof(DmaBufferInfo));
    size_t bpp_mesh_merged = 8 * sizeof(float) * 2;
    alloc_drm_buf(&dmaBufMesh, mesh.u32Width, mesh.u32Height, bpp_mesh_merged);
    mesh.fd[0]      = dmaBufMesh.fd;
    mesh.virAddr[0] = dmaBufMesh.ptr;
    size_t meshSize = mesh.u32Width * mesh.u32Height * sizeof(float) * 2;
    ret |= readBinaryFile(meshxy_path, mesh.virAddr[0], meshSize);
    if (ret != 0) {
        printf("read mesh data failed.\n");
        return ;
    }

    /* Step 2-4. Config Op Attr. */
    RKALG_LUT_OP_ATTR_S stOpAttr;
    memset(&stOpAttr, 0, sizeof(RKALG_LUT_OP_ATTR_S));

    /* Step 2-5. Call doLut API */
    RKALG_LUT_TASK_S stTask;
    stTask.pSrcImage = &srcImage;
    stTask.pDstImage = &dstImage;
    stTask.pMesh     = &mesh;
    stTask.pOpAttr   = &stOpAttr; // only take effect when mode is
RKALG_IDC_LUT_MANUAL_MODE.
    RKALG_IDC_LUT_DoLut(&stLutCtx, &stTask);

    /* Step 2-6. Write dstImg to file */
    FILE *fp_out = fopen(dst_img_path, "wb");
    if (fp_out == NULL) {
        printf("Can not open %s\n", dst_img_path);
        return;
    }
```

```
    unsigned char* pt     = NULL;
    unsigned char* pDstY  = (unsigned char*)dmaBufDst.ptr;
    unsigned char* pDstUV = pDstY + dstImage.u32Stride[0] * dstImage.u32HgtStride[0];

    for (size_t row_idx = 0; row_idx < dstImage.u32Height; row_idx++) {
        pt = pDstY + row_idx * dstImage.u32Stride[0];
        fwrite(pt, sizeof(unsigned char), dstImage.u32Width, fp_out);
    }
    for (size_t row_idx = 0; row_idx < dstImage.u32Height / 2; row_idx++) {
        pt = pDstUV + row_idx * dstImage.u32Stride[0];
        fwrite(pt, sizeof(unsigned char), dstImage.u32Width, fp_out);
    }
    fclose(fp_out);
    printf("write %s success.\n", dst_img_path);

    /* Step 3. Call deInit API */
    RKALG_IDC_LUT_Deinit(&stLutCtx);
    if (dmaBufMesh.ptr != nullptr) {
        drm_munmap(dmaBufMesh.ptr, dmaBufMesh.size);
        dmaBufMesh.ptr = nullptr;
    }
    close(dmaBufMesh.fd);
    if (dmaBufSrc.ptr != nullptr) {
        drm_munmap(dmaBufSrc.ptr, dmaBufSrc.size);
        dmaBufSrc.ptr = nullptr;
    }
    close(dmaBufSrc.fd);
    if (dmaBufDst.ptr != nullptr) {
        drm_munmap(dmaBufDst.ptr, dmaBufDst.size);
        dmaBufDst.ptr = nullptr;
    }
    close(dmaBufDst.fd);

    printf("test end.\n");
}
```